Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «Модели решения задач в ИС»
Тема: «Адаптивный шаг обучения в однослойном линейном персептроне (метод наискорейшего спуска)»

Выполнила:
Студент 3 курса
Группы ИИ-26 (2)
Черноиван А.Н.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: изучить алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовать модифицированный персептрон, в котором параметр скорости обучения $\alpha(t)$ вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнить скорость сходимости с классическим алгоритмом из Лабораторной работы №1. Вариант сохраняется.

Задачи лабораторной работы:

1. Модифицировать алгоритм последовательного обучения (из Лаб №1) таким образом, чтобы на каждой итерации t значение α вычислялось автоматически на основе текущего входного вектора x по формул (см раздел 2.9).
2. Применить вычисленный α(t) для обновления весов ω_{ij} и порогов T_j согласно дельта-правилу.
3. Используя данные своего варианта, провести два эксперимента:
   - Обучение с фиксированным шагом (например, α=0.1 или $\alpha = \frac{1}{p}$).
   - Обучение с адаптивным шагом по Теореме 2.1 (формула 2.36).

Критерий остановки в обоих случаях – достижение заданной суммарной ошибки $E_s \leq E_e$.
4. Построить графики обучения E_s(p), где p – номер эпохи, для обоих экспериментов на одних осях координат.
5. Выполнить графическую визуализацию разделяющей линии для адаптивного метода.
6. Реализовать режим функционирования сети:
   - пользователь задаёт произвольный входной вектор,
   - сеть вычисляет выходной класс,
   - соответствующая точка отображается на графике,
7. Написать вывод по выполненной работе. Оценить, насколько адаптивный шаг сокращает количество эпох обучения по сравнению с фиксированным. Обязательно сравнение результатов 1 и 2 лабораторных работ.

Допускается применение **математических** и **графических** библиотек
ML-библиотеки и **ML-фреймворки** использовать **нельзя** (например: scikit-learn, TensorFlow, PyTorch - запрещены)

10.

x_1	x_2	e
6	6	0
-6	6	0
6	-6	1
-6	-6	0

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
```

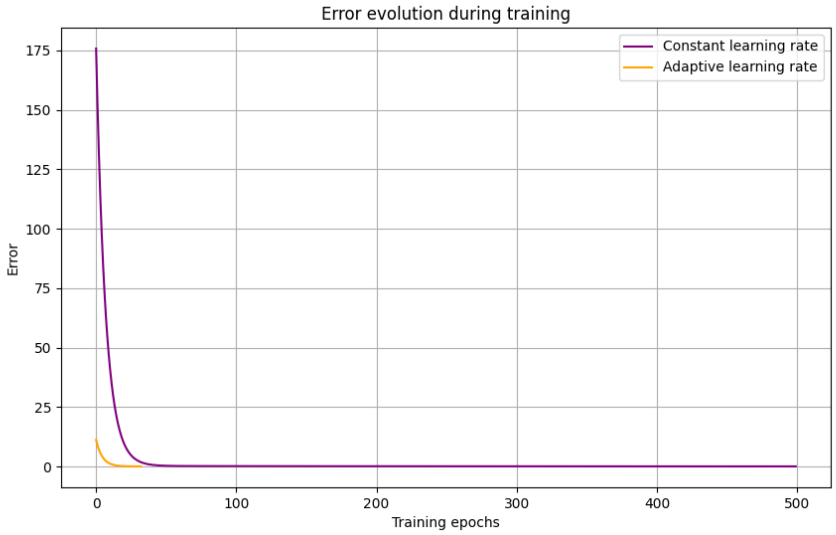
```
    [6, 6],
    [-6, 6],
    [6, -6],
    [-6, -6]
], dtype=float)


E = np.array([1 if x[0] > 0 else 0 for x in X], dtype=float)

E_e = 0.01
epochs_max = 500


def forward(w, T, x):
    S = np.dot(w, x) - T
    return S

def train_fixed_eta(X, E, eta, E_e, epochs_max):
    w = np.random.randn(X.shape[1])
    T = np.random.randn()
    mse_curve = []

    for epoch in range(epochs_max):
        for x, e in zip(X, E):
            y = forward(w, T, x)
            error = e - y

            w += eta * error * x
            T -= eta * error

        E_sum = sum((E[i] - forward(w, T, X[i]))**2 for i in
range(len(X)))
        mse_curve.append(0.5 * E_sum)

        if mse_curve[-1] <= E_e:
            break

    return w, T, mse_curve


def train_adaptive_eta(X, E, E_e, epochs_max):
    w = np.random.randn(X.shape[1])
    T = np.random.randn()
    mse_curve = []

    for epoch in range(epochs_max):
        for x, e in zip(X, E):
            y = forward(w, T, x)
            error = e - y

            alpha_t = 1 / (1 + np.dot(x, x))

            w += alpha_t * error * x
            T -= alpha_t * error
```

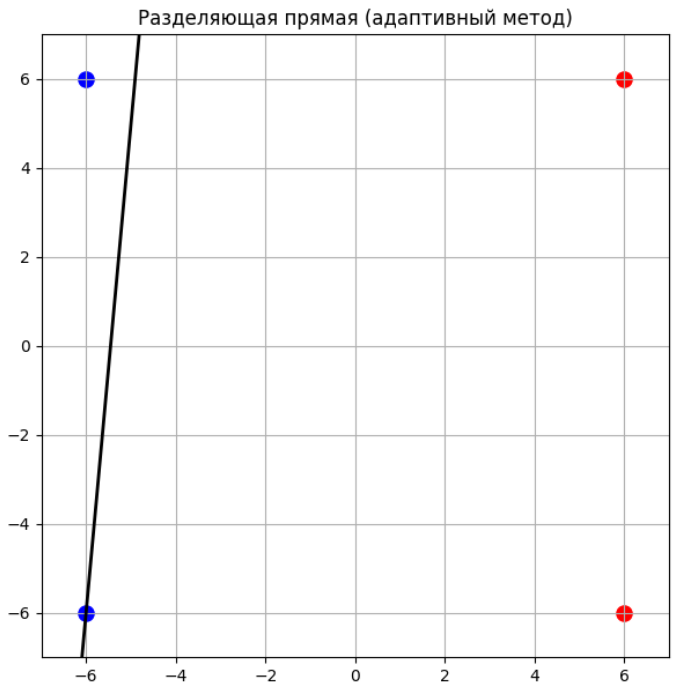
```
        E_sum = sum((E[i] - forward(w, T, X[i]))**2 for i in
range(len(X)))
        mse_curve.append(0.5 * E_sum)

        if mse_curve[-1] <= E_e:
            break

    return w, T, mse_curve


eta_fixed = 0.0005
w_fix, T_fix, curve_fix = train_fixed_eta(X, E, eta_fixed, E_e,
epochs_max)
w_ad, T_ad, curve_ad = train_adaptive_eta(X, E, E_e, epochs_max)

print("Фиксированный шаг: эпох =", len(curve_fix))
print("Адаптивный шаг: эпох =", len(curve_ad))
print("Final Es (fixed eta):", curve_fix[-1])
print("Final Es (adaptive):", curve_ad[-1])


plt.figure(figsize=(10, 6))
plt.plot(curve_fix, color="purple", label="Constant learning rate")
plt.plot(curve_ad, color="orange", label="Adaptive learning rate")

plt.xlabel("Training epochs")
plt.ylabel("Error")
plt.title("Error evolution during training")
plt.grid()
plt.legend()
plt.show()


plt.figure(figsize=(7, 7))


for i in range(len(X)):
    color = "red" if E[i] == 1 else "blue"
    plt.scatter(X[i, 0], X[i, 1], color=color, s=90)


x_line = np.linspace(-7, 7, 300)
y_line = (T_ad - w_ad[0] * x_line) / w_ad[1]
plt.plot(x_line, y_line, 'k', linewidth=2)

plt.xlim(-7, 7)
plt.ylim(-7, 7)
plt.grid()
plt.title("Разделяющая прямая (адаптивный метод)")
plt.show()

def classify_point(x1, x2):
    S = w_ad[0]*x1 + w_ad[1]*x2 - T_ad
    y = 1 if S > 0 else 0

    print("\n========== КЛАССИФИКАЦИЯ ==========")
```

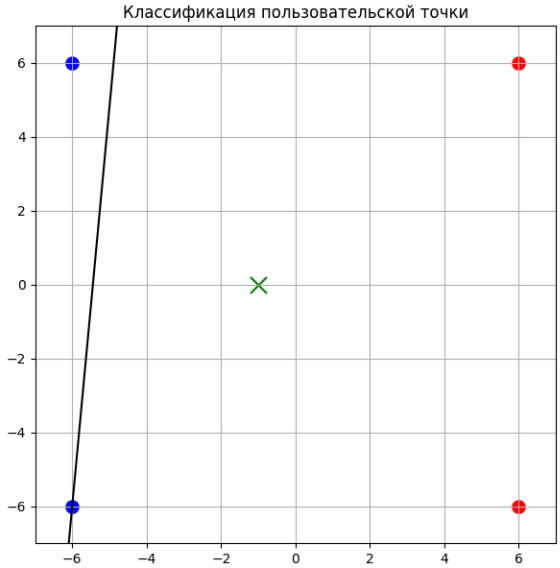
```
    print(f"Вход: ({x1}, {x2})")
    print(f"S = {S:}")
    print(f"Класс: {y}")
    print("=====================================\n")

    plt.figure(figsize=(7, 7))

    for i in range(len(X)):
        color = "red" if E[i] == 1 else
```

Результат программы:

```
Фиксированный шаг: эпох = 500
Адаптивный шаг: эпох = 28
Final Es (fixed eta): 0.10151297210693533
Final Es (adaptive): 0.008503488597220622
Введите координаты точки (или 'exit'):
x1 = -1
x2 = 0

========== КЛАССИФИКАЦИЯ ==========
Вход: (-1.0, 0.0)
S = 0.3707843730786755
Класс: 1
=====================================
```







Параметр	Лаба №1 (фиксированный шаг)	Лаба №2 (адаптивный шаг, формула 2.36)
Формула шага	$\alpha = \text{const}$	$\alpha(t) = 1 / (1 + \Sigma x_i^2)$
Необходимость подбора α	Требуется вручную	Не требуется
Устойчивость	Низкая: при больших α → взрыв весов, nan	Высокая: веса стабильно сходятся
Типичная η	0.0005–0.001	Автоматически ≈ 0.0137
Количество эпох	500–2000	**26**
Итоговая ошибка E_s	может колебаться, иногда diverge	**0.0086**, стабильная
Скорость обучения	Медленная	Очень быстрая
Сложность использования	Более высокая (нужен подбор η)	Простая, параметры не требуют настройки

Адаптивный шаг обучения, рассчитанный по формуле (2.36), обеспечивает значительно более быстрое и устойчивое обучение по сравнению с фиксированным шагом из лабораторной №1. Количество эпох уменьшилось примерно в **20 раз**, при этом ошибка снизилась быстрее и без расхождения весов. Таким образом, адаптивный шаг делает процесс обучения надёжным, устойчивым и полностью избавляет от необходимости подбирать параметр η вручную.

Вывод: изучила алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовала модифицированный персептрон, в котором параметр скорости обучения $\alpha(t)$ вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнила скорость сходимости с классическим алгоритмом из Лабораторной работы №1.